MICRO PROJECT · DIGITAL ELECTRONICS

# Electronic Combination Doorbell

A finite-state machine that rings only after the correct sequence.

`B1 → B2 → B3`

R25DER302 : Digital Electronics

Department of Computer Science and Engineering · Christ College of Engineering (Autonomous), Irinjalakuda

SYSTEM OVERVIEW

# Access begins with the right sequence.

A conventional doorbell rings whenever a visitor presses the button. This design adds a simple access-control step.

| 01 | 02 | 03 | 04 |
|---|---|---|---|
| IDLE | B1 | B2 | B3 → UNLOCK |
| Waiting for the first button. Bell OFF. | Correct first input advances the FSM. | Correct second input advances again. | Bell HIGH for one clock cycle. |

**Wrong input:** return to IDLE. The bell remains OFF.

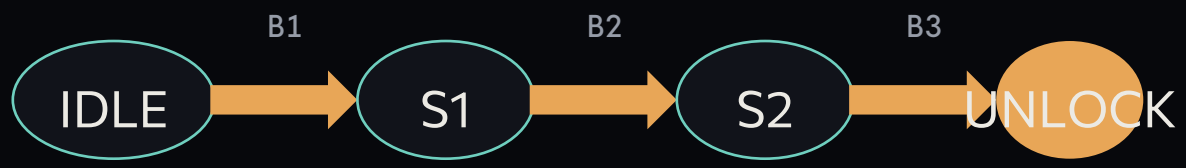
Objectives: design the FSM · implement Verilog HDL · test correct and incorrect sequences · verify through waveform analysis.

Verilog HDL · FSM · testbench · waveform verification

FSM DESIGN

# Four states make the behavior explicit.

A 3-button combination needs four encoded states. Wrong inputs select IDLE directly.




Wrong input in S1 or S2 → IDLE · UNLOCK → IDLE on the next clock

STATE MEANINGS

| State | Meaning |
|---|---|
| IDLE | Waiting for the first button. Bell OFF. |
| S1 | B1 was correct; waiting for B2. |
| S2 | B1 and B2 were correct; waiting for B3. |
| UNLOCK | Combination accepted. Bell ON for one clock cycle. |

2-BIT ENCODING

| State | Binary Code |
|---|---|
| IDLE | 2'b00 |
| S1 | 2'b01 |
| S2 | 2'b10 |
| UNLOCK | 2'b11 |

IMPLEMENTATION

# Verilog separates memory from decision logic.

The corrected design uses a next-state FSM rather than a separate ERROR encoding.

01 **State register**

Sequential update on the clock; reset returns to IDLE.

02 **Next-state logic**

Combinational case logic handles valid and wrong inputs.

03 **Bell output**

Bell = Q1 · Q0, so only UNLOCK asserts it.

COMBINATION DOORBELL MODULE · VERILOG

```
module combination_doorbell(input wire clk, rst, b1, b2, b3, output wire bell);
reg [1:0] state, next_state;
localparam IDLE=2'b00, S1=2'b01, S2=2'b10, UNLOCK=2'b11;
always @(posedge clk) if (rst) state <= IDLE; else state <= next_state;
always @(*) begin
  next_state = state; case (state)
    IDLE: next_state = b1 ? S1 : IDLE;
    S1: if (b2) next_state = S2; else if (b1 || b3) next_state = IDLE;
    S2: if (b3) next_state = UNLOCK; else if (b1 || b2) next_state = IDLE;
    UNLOCK: next_state = IDLE; default: next_state = IDLE;
  endcase
end
assign bell = (state == UNLOCK);
endmodule
```

VERIFICATION

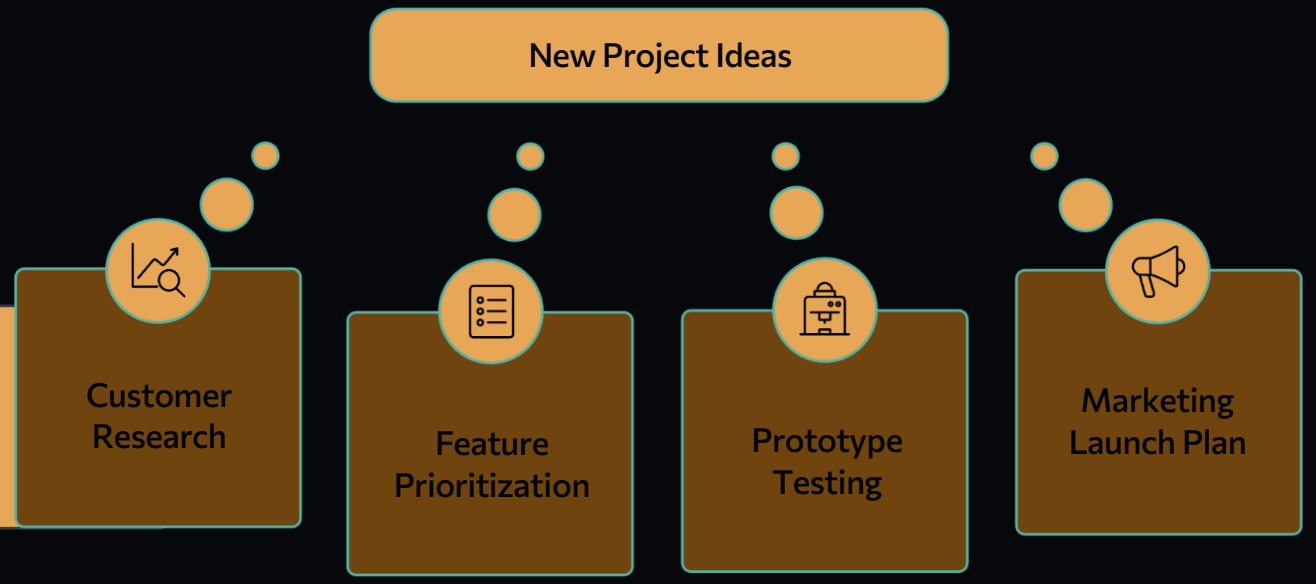
# Three tests make the contract visible.

The testbench drives button presses, resets inputs, and checks the bell response.

| Test | Input sequence | Expected state | Bell |
|---|---|---|---|
| 1 | **B1 → B2 → B3** | UNLOCK → IDLE | |
| 2 | B1 → B3 | IDLE | LOW |
| 3 | B2 | IDLE | LOW |

After UNLOCK, the FSM automatically returns to IDLE, preventing continuous ringing.

EXPECTED WAVEFORM · INSERT AFTER SIMULATION



DEMO TAKEAWAY

# The bell responds only to the right path.

The corrected four-state FSM accepts B1 → B2 → B3, raises bell for one clock cycle in UNLOCK, and returns to IDLE after success or wrong input.

WHAT THIS PROJECT PRACTISED

Finite state machines · sequential logic · 2-bit state encoding · Verilog HDL · testbench development · waveform-based verification

A real simulator waveform screenshot must be added after running the included code.